Integrantes:

- Soledad Faundez Labra,
- Esteban Olivares Cabrera,
- Martin Osorio Martinez,
- Renato Espina Cid,
- Carlo Reyes Espinoza,
- Ignacia Reyes Espinoza.

Asignatura:

- Deep Learning.

Docente:

- Marcelo Tapia Contreras.

Introducción:

En el presente trabajo se aplicarán los fundamentos de Deep Learning mediante la implementación de una red neuroal artificial de tipo *Multilayer Perceptron (MLP)* para resolver un problema de clasificación de imágenes. Para ello, se utiliza el data set **Fashion-MNIST**, el cual contiene imágenes en escala de grises de 28x28 píxeles correspondientes a distintas categorías de prendas de vestir, tales como camisetas, pantalones, zapatillas, entre otros.

Objetivo principal:

Desarrollar un modelo capaz de aprender patrones a partir de los datos y clasificar correctamente nuevas imágenes. Para lograrlo, se realiza un proceso completo que abarca desde la carga y preprocesamiento de los datos hasta la evaluación del modelo utilziando diversas métricas de desempeño.

En una primera etapa, se preparan los datos para su uso en redes neuroales, transformando las imágenes a vectores y normalizando sus valores para mejorar la eficiencia del entrenamiento. Posteriormente, se construye una red neuronal multicapa, definiendo su arquitectura, funciones de activación y función de pérdida.

A continuación, se entrenará el modelo variando distintos hiperparámetros claves, tales como el número de épocas, tamaño de batch y optimizadores. Se realizará una comparación entre distintos algoritmos de optimización, incluyendo *SGD, Adam, RMSprop*, así como variantes como *Momentum y Nesterov*, con el fin de identificar cuál ofrece un mejor desempeño.

Además, se incorporarán técnicas de regularización como *Dropout y Early Stopping*, con el objetivo de reducir el sobreajuste(*overfitting*) y mejorar la capacidad de generalización del modelo.

Finalmente, el desempeño del modelo será evaluado mediante métricas como *accuracy, precision, recall y F1-score*, permitiendo analizar de manera integral la calidad de las predicciones. A partir de estos resultados, se jusatificarán las decisiones tomadas durante el desarrollo y se seleccionará la mejor configuración del modelo, mostrando ejemplos visuales de cómo responde el mejor modelo obtenido.

PARTE 1: Carga de Datos.

```
# import de librerias
from tensorflow import keras
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random

# Semillas para reproducibilidad
np.random.seed(42)
tf.random.set_seed(42)
random.seed(42)

# Cargar dataset Fashion-MNIST
(X_train, y_train), (X_test, y_test) =
keras.datasets.fashion_mnist.load_data()

# Mostrar dimensiones
print("Train:", X_train.shape, y_train.shape)
print("Test:", X_test.shape, y_test.shape)
```

```
Train: (60000, 28, 28) (60000,)
Test: (10000, 28, 28) (10000,)
```

```
# Nombres de clases
clases = ['Camiseta', 'Pantalón', 'Suéter', 'Vestido', 'Abrigo',
          'Sandalia', 'Camisa', 'Zapatilla', 'Bolso', 'Botín']

# Mostrar ejemplos
plt.figure(figsize=(10,10))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title(clases[y_train[i]])
```

```
    plt.axis('off')
plt.show()
```

Se utilizó el dataset Fashion-MNIST disponible en Keras, el cual contiene imágenes en escala de grises de 28x28 píxeles correspondientes a distintas categorías de prendas de vestir.

El conjunto de datos está dividido en 60.000 imágenes para entrenamiento y 10.000 para prueba. Cada imagen está etiquetada en una de las 10 clases disponibles, como camisetas, pantalones, zapatos, entre otros.

Se visualizaron algunos ejemplos del dataset para comprender la estructura de los datos y las clases asociadas.

PARTE 2: Preprocesamiento de Datos.

```
# Normalizar los datos (0–255 → 0–1)
X_train = X_train / 255.0
X_test = X_test / 255.0

# Aplanar las imágenes (28x28 → 784)
X_train = X_train.reshape(-1, 28*28)
X_test = X_test.reshape(-1, 28*28)

Y_train = y_train
Y_test = y_test

# Verificamos dimensiones
print("X_train:", X_train.shape)
print("Y_train:", Y_train.shape)
```

```
X_train: (60000, 784)
Y_train: (60000,)
```

Para preparar los datos para el entrenamiento de la red neuronal, se realizaron varias transformaciones.

Primero, se normalizaron los valores de los píxeles dividiéndolos por 255, con el objetivo de llevarlos a un rango entre 0 y 1. Esto mejora la estabilidad del entrenamiento y permite una mejor convergencia del modelo.

Luego, las imágenes de 28x28 píxeles fueron transformadas a vectores unidimensionales de 784 elementos, ya que las redes MLP requieren entradas en formato vectorial.

Las etiquetas se representaron como números enteros del 0 al 9, donde cada número corresponde a una clase. Para que el modelo pueda trabajar con este formato, se utilizó la función de pérdida sparse categorical crossentropy, que permite entrenar directamente con etiquetas numéricas sin necesidad de codificación adicional.

PARTE 3: Modelo MLP.

```
# setamos los seed para números random:

np.random.seed(30)
tf.random.set_seed(30)
```

```python
# Definimos las dimensiones de entrada y salida
N = X_train.shape[1]  # Dimensiones de entrada (784 píxeles)
C = 10 # Número de clases de salida (10 clases)

# Creación de la red:

h1 = 512
h2 = 256

def crear_modelo():
    model = keras.Sequential([
        keras.layers.Dense(h1, input_dim=N,  activation='relu'),
        keras.layers.Dense(256, activation='relu'),
        keras.layers.Dense(256, activation='relu'),
        keras.layers.Dense(10, activation='softmax')
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.001),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

modelA = crear_modelo()
modelA.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

```
Model: "sequential"
```

```
┏━━━
```

```
65,792 │
├──────────────────────────────────────┼──────────────────────────────┼──────────
──────┤
│ dense_3 (Dense)                      │ (None, 10)                   │
2,570 │
└──────────────────────────────────────┴──────────────────────────────┴──────────
──────┘

 Total params: 601,610 (2.29 MB)

 Trainable params: 601,610 (2.29 MB)

 Non-trainable params: 0 (0.00 B)
```

El modelo implementado corresponde a una red neuronal multicapa (MLP) compuesta por tres capas ocultas con 512, 256 y 256 neuronas respectivamente, y una capa de salida de 10 neuronas.

El modelo presenta un total de 601.610 parámetros entrenables, lo que indica una alta capacidad de aprendizaje. Esta gran cantidad de parámetros permite al modelo capturar patrones complejos presentes en las imágenes del dataset.

Sin embargo, esta alta capacidad también implica un mayor riesgo de sobreajuste, ya que el modelo puede llegar a memorizar los datos de entrenamiento en lugar de generalizar correctamente.

El learning rate corresponde a la tasa de aprendizaje del modelo, es decir, la magnitud de los ajustes realizados en los pesos durante el entrenamiento. Se utilizó un valor de 0.001, el cual corresponde al valor por defecto del optimizador Adam, ya que permite un equilibrio adecuado entre velocidad de convergencia y estabilidad del entrenamiento.

Por esta razón, este modelo será utilizado como base para posteriormente aplicar técnicas de regularización que permitan mejorar su desempeño en datos no vistos.

PARTE 4: Entrenamiento y Visualización.

4.1: Entrenamiento A:

Se decidió entrenar el *modelA* con **30 épocas** y un **batch size de 32**, debido a que es una configuración estándar para tener una primera exploración para observar el comportamiento del modelo sin inucrrir en tiempos de entrenamiento excesivos.

```
# ===== ENTRENAMIENTO A =====
np.random.seed(30)
```

```
tf.random.set_seed(30)

modelA = crear_modelo()

historyA = modelA.fit(
    X_train, Y_train,
    epochs=30,
    batch_size=32,
    validation_data=(X_test, Y_test)
)
```

```
Epoch 1/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 32s 15ms/step - accuracy: 0.8232 -
loss: 0.4808 - val_accuracy: 0.8380 - val_loss: 0.4477
Epoch 2/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 25s 13ms/step - accuracy: 0.8670 -
loss: 0.3625 - val_accuracy: 0.8497 - val_loss: 0.4162
Epoch 3/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 43s 14ms/step - accuracy: 0.8802 -
loss: 0.3221 - val_accuracy: 0.8619 - val_loss: 0.3860
Epoch 4/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 25s 14ms/step - accuracy: 0.8899 -
loss: 0.2967 - val_accuracy: 0.8582 - val_loss: 0.4043
Epoch 5/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 25s 13ms/step - accuracy: 0.8951 -
loss: 0.2782 - val_accuracy: 0.8649 - val_loss: 0.4023
Epoch 6/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 23s 12ms/step - accuracy: 0.9010 -
loss: 0.2613 - val_accuracy: 0.8745 - val_loss: 0.3788
Epoch 7/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 24s 13ms/step - accuracy: 0.9063 -
loss: 0.2486 - val_accuracy: 0.8738 - val_loss: 0.3808
Epoch 8/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 24s 13ms/step - accuracy: 0.9103 -
loss: 0.2363 - val_accuracy: 0.8743 - val_loss: 0.4107
Epoch 9/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 25s 13ms/step - accuracy: 0.9126 -
loss: 0.2274 - val_accuracy: 0.8812 - val_loss: 0.3789
Epoch 10/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 24s 13ms/step - accuracy: 0.9161 -
loss: 0.2201 - val_accuracy: 0.8868 - val_loss: 0.3729
Epoch 11/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 24s 13ms/step - accuracy: 0.9183 -
loss: 0.2120 - val_accuracy: 0.8826 - val_loss: 0.4023
Epoch 12/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 22s 12ms/step - accuracy: 0.9211 -
loss: 0.2036 - val_accuracy: 0.8841 - val_loss: 0.3871
Epoch 13/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 23s 12ms/step - accuracy: 0.9255 -
loss: 0.1960 - val_accuracy: 0.8911 - val_loss: 0.3812
```

```
Epoch 14/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 23s 12ms/step - accuracy: 0.9265 -
loss: 0.1915 - val_accuracy: 0.8849 - val_loss: 0.4320
Epoch 15/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 24s 13ms/step - accuracy: 0.9280 -
loss: 0.1854 - val_accuracy: 0.8860 - val_loss: 0.4080
Epoch 16/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 22s 12ms/step - accuracy: 0.9302 -
loss: 0.1789 - val_accuracy: 0.8869 - val_loss: 0.4531
Epoch 17/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 23s 12ms/step - accuracy: 0.9322 -
loss: 0.1772 - val_accuracy: 0.8857 - val_loss: 0.4701
Epoch 18/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 42s 13ms/step - accuracy: 0.9339 -
loss: 0.1698 - val_accuracy: 0.8894 - val_loss: 0.4415
Epoch 19/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 44s 14ms/step - accuracy: 0.9349 -
loss: 0.1682 - val_accuracy: 0.8837 - val_loss: 0.4875
Epoch 20/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 22s 12ms/step - accuracy: 0.9385 -
loss: 0.1601 - val_accuracy: 0.8892 - val_loss: 0.4606
Epoch 21/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 21s 11ms/step - accuracy: 0.9382 -
loss: 0.1593 - val_accuracy: 0.8877 - val_loss: 0.4585
Epoch 22/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 23s 12ms/step - accuracy: 0.9406 -
loss: 0.1538 - val_accuracy: 0.8884 - val_loss: 0.5126
Epoch 23/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 39s 11ms/step - accuracy: 0.9410 -
loss: 0.1537 - val_accuracy: 0.8844 - val_loss: 0.5559
Epoch 24/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 23s 12ms/step - accuracy: 0.9416 -
loss: 0.1516 - val_accuracy: 0.8870 - val_loss: 0.5048
Epoch 25/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 22s 12ms/step - accuracy: 0.9430 -
loss: 0.1467 - val_accuracy: 0.8905 - val_loss: 0.5559
Epoch 26/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 22s 12ms/step - accuracy: 0.9454 -
loss: 0.1409 - val_accuracy: 0.8894 - val_loss: 0.5504
Epoch 27/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 23s 12ms/step - accuracy: 0.9460 -
loss: 0.1416 - val_accuracy: 0.8845 - val_loss: 0.6364
Epoch 28/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 23s 12ms/step - accuracy: 0.9473 -
loss: 0.1382 - val_accuracy: 0.8848 - val_loss: 0.5754
Epoch 29/30
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 40s 12ms/step - accuracy: 0.9471 -
loss: 0.1373 - val_accuracy: 0.8912 - val_loss: 0.6326
Epoch 30/30
```

```
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 23s 12ms/step - accuracy: 0.9491 -
loss: 0.1315 - val_accuracy: 0.8859 - val_loss: 0.6434
```

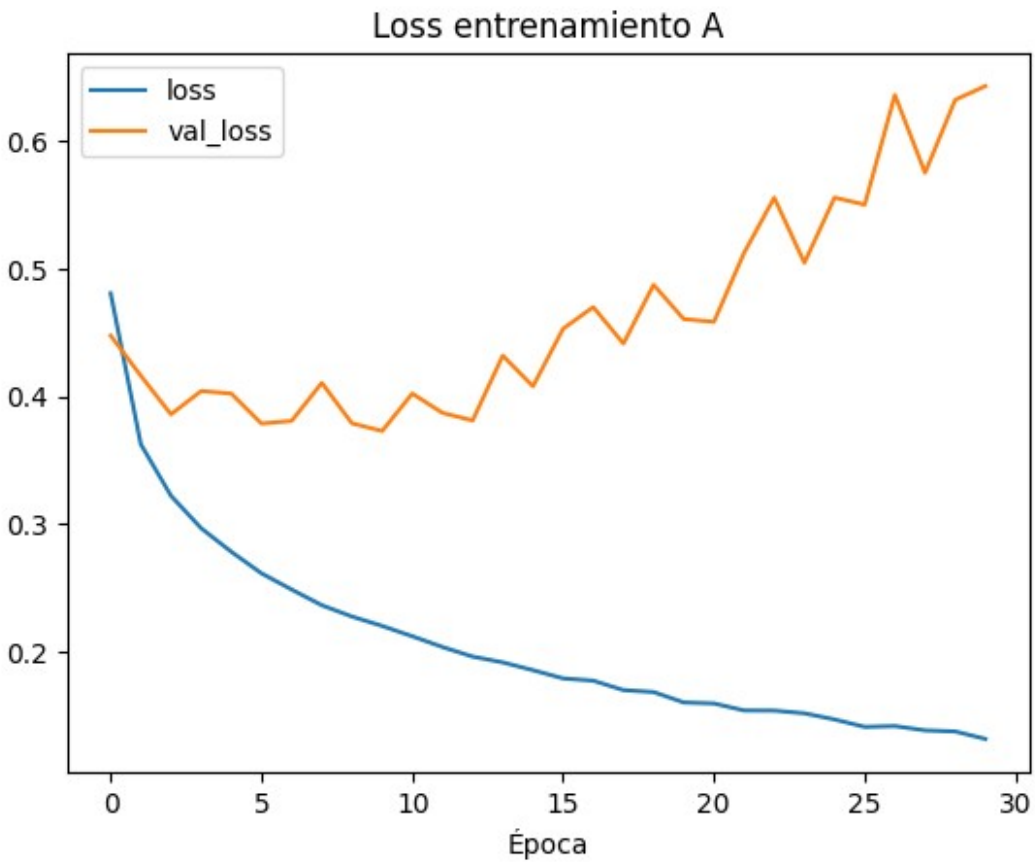
```python
# Resultados modelA:

print(f"Train accuracy \u2248 {historyA.history['accuracy'][-
1]*100:.1f}%")
print(f"Val accuracy \u2248 {historyA.history['val_accuracy'][-
1]*100:.1f}%")
```

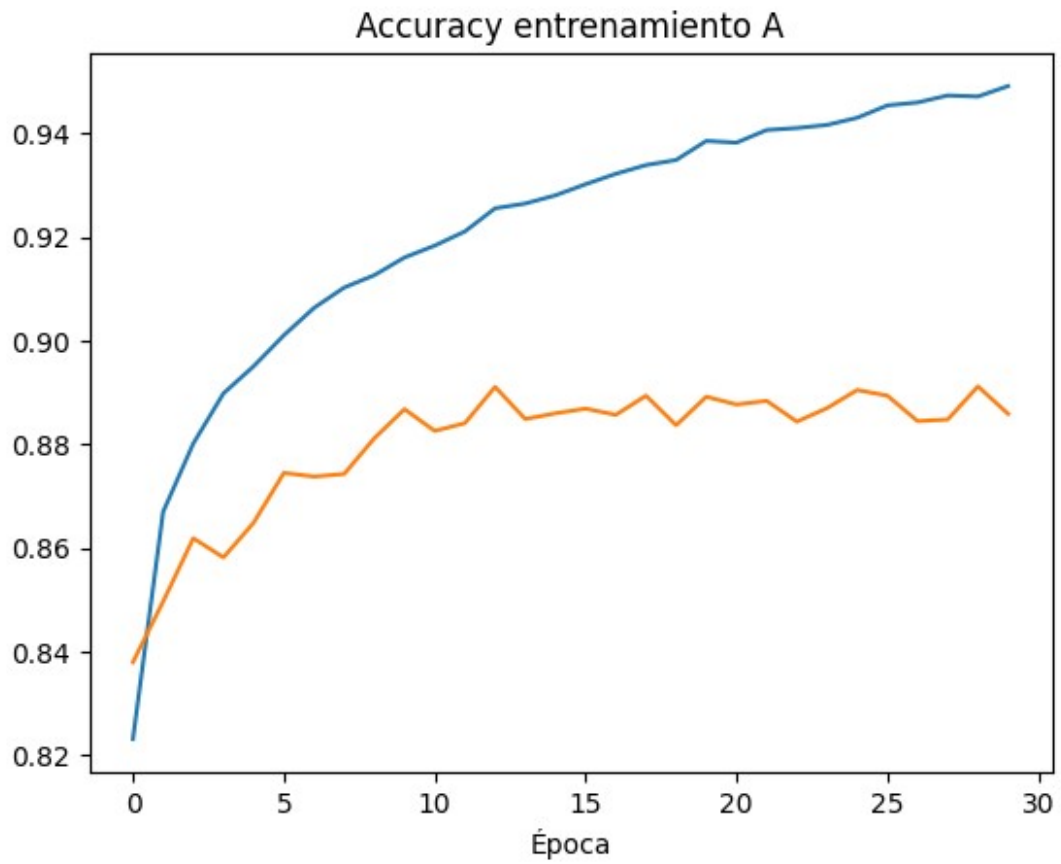
```
Train accuracy ≈ 94.9%
Val accuracy ≈ 88.6%
```

4.2: Visualización entrenamiento A:

Gráfico de Loss y Accuracy:

```python
# Loss
plt.plot(historyA.history['loss'], label='loss')
plt.plot(historyA.history['val_loss'], label='val_loss')
plt.title('Loss entrenamiento A')
plt.xlabel('Época')
plt.legend()
plt.show()

# Accuracy
plt.plot(historyA.history['accuracy'], label='accuracy')
plt.plot(historyA.history['val_accuracy'], label='val_accuracy')
plt.title('Accuracy entrenamiento A')
plt.xlabel('Época')
plt.show()
```


Loss entrenamiento A
loss
val_loss
0.6
0.5
0.4
0.3
0.2
0
5
10
15
20
25
30
Época



Pérdida

- La **pérdida de entrenamiento** disminuye de forma constante a lo largo de las épocas, indicando que el modelo está aprendiendo correctamente los patrones de datos de entrenamiento.
- Por otro lado, la **pérdida de validación** disminuye inicialmente, pero luego comienza a aumentar progresivamente, lo que es una señal clara de **sobreajuste**.

Acierto

- El **accuracy de entrenamiento** aumentaa de forma sostenida, alcanzando valores cercanos al 95%. Esto indica que el modelo se ajusta muy bien a los datos que ha visto.
- El **accuracy de validación** mejora al inicio pero luego se estanca e inluso presenta pequeñas fluctuaciones. Este becha creciente entrenamiento y validación refuerza la presencia de **overfitting**.

4.3: Entrenamiento B:

Ante la evidencia del entrenamiento anterior, se tomó la decisión de aumentar los hiperparámetros por **50 épocas** y un **batch size de 128**, lo que permite analizar cómo del modelo se comporta bajo una configuración más intensiva.

```
# ===== ENTRENAMIENTO B =====
np.random.seed(30)
tf.random.set_seed(30)

modelB = crear_modelo()

historyB = modelB.fit(
    X_train, Y_train,
    epochs=50,
    batch_size=128,
    validation_data=(X_test, Y_test)
)
```

```
Epoch 1/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 10s 18ms/step - accuracy: 0.8197 - loss:
0.5001 - val_accuracy: 0.8408 - val_loss: 0.4359
Epoch 2/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 15ms/step - accuracy: 0.8672 - loss:
0.3596 - val_accuracy: 0.8541 - val_loss: 0.4031
Epoch 3/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.8806 - loss:
0.3214 - val_accuracy: 0.8679 - val_loss: 0.3820
Epoch 4/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.8896 - loss:
0.2952 - val_accuracy: 0.8744 - val_loss: 0.3601
Epoch 5/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 15ms/step - accuracy: 0.8969 - loss:
0.2760 - val_accuracy: 0.8753 - val_loss: 0.3510
Epoch 6/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.9035 - loss:
0.2587 - val_accuracy: 0.8756 - val_loss: 0.3708
Epoch 7/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.9069 - loss:
0.2485 - val_accuracy: 0.8755 - val_loss: 0.3696
Epoch 8/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 8s 16ms/step - accuracy: 0.9115 - loss:
0.2362 - val_accuracy: 0.8764 - val_loss: 0.3574
Epoch 9/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.9149 - loss:
0.2252 - val_accuracy: 0.8781 - val_loss: 0.3696
Epoch 10/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9178 - loss:
0.2181 - val_accuracy: 0.8809 - val_loss: 0.3785
```

```
Epoch 11/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - accuracy: 0.9206 - loss:
0.2072 - val_accuracy: 0.8799 - val_loss: 0.4131
Epoch 12/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 18ms/step - accuracy: 0.9247 - loss:
0.1991 - val_accuracy: 0.8791 - val_loss: 0.4152
Epoch 13/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 8s 17ms/step - accuracy: 0.9269 - loss:
0.1929 - val_accuracy: 0.8790 - val_loss: 0.4164
Epoch 14/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 8s 16ms/step - accuracy: 0.9301 - loss:
0.1822 - val_accuracy: 0.8828 - val_loss: 0.4149
Epoch 15/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 8s 18ms/step - accuracy: 0.9329 - loss:
0.1750 - val_accuracy: 0.8847 - val_loss: 0.4338
Epoch 16/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9350 - loss:
0.1692 - val_accuracy: 0.8836 - val_loss: 0.4256
Epoch 17/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 18ms/step - accuracy: 0.9387 - loss:
0.1602 - val_accuracy: 0.8815 - val_loss: 0.4646
Epoch 18/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.9414 - loss:
0.1532 - val_accuracy: 0.8818 - val_loss: 0.5140
Epoch 19/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 15ms/step - accuracy: 0.9428 - loss:
0.1480 - val_accuracy: 0.8801 - val_loss: 0.5144
Epoch 20/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.9437 - loss:
0.1445 - val_accuracy: 0.8791 - val_loss: 0.4761
Epoch 21/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 20ms/step - accuracy: 0.9485 - loss:
0.1339 - val_accuracy: 0.8844 - val_loss: 0.4739
Epoch 22/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 8s 17ms/step - accuracy: 0.9503 - loss:
0.1296 - val_accuracy: 0.8808 - val_loss: 0.5096
Epoch 23/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.9496 - loss:
0.1292 - val_accuracy: 0.8838 - val_loss: 0.4949
Epoch 24/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 18ms/step - accuracy: 0.9510 - loss:
0.1253 - val_accuracy: 0.8866 - val_loss: 0.4714
Epoch 25/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 17ms/step - accuracy: 0.9527 - loss:
0.1213 - val_accuracy: 0.8785 - val_loss: 0.5516
Epoch 26/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 18ms/step - accuracy: 0.9550 - loss:
0.1189 - val_accuracy: 0.8830 - val_loss: 0.5239
Epoch 27/50
```

```
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.9579 - loss:
0.1069 - val_accuracy: 0.8847 - val_loss: 0.5310
Epoch 28/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 16ms/step - accuracy: 0.9589 - loss:
0.1049 - val_accuracy: 0.8915 - val_loss: 0.5185
Epoch 29/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - accuracy: 0.9588 - loss:
0.1043 - val_accuracy: 0.8817 - val_loss: 0.5450
Epoch 30/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 8s 18ms/step - accuracy: 0.9596 - loss:
0.1011 - val_accuracy: 0.8884 - val_loss: 0.5455
Epoch 31/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 8s 16ms/step - accuracy: 0.9618 - loss:
0.0973 - val_accuracy: 0.8831 - val_loss: 0.5625
Epoch 32/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - accuracy: 0.9633 - loss:
0.0955 - val_accuracy: 0.8828 - val_loss: 0.5970
Epoch 33/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 18ms/step - accuracy: 0.9643 - loss:
0.0927 - val_accuracy: 0.8862 - val_loss: 0.5829
Epoch 34/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 10s 18ms/step - accuracy: 0.9656 - loss:
0.0878 - val_accuracy: 0.8884 - val_loss: 0.6069
Epoch 35/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 15ms/step - accuracy: 0.9652 - loss:
0.0883 - val_accuracy: 0.8906 - val_loss: 0.5862
Epoch 36/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.9679 - loss:
0.0850 - val_accuracy: 0.8889 - val_loss: 0.6392
Epoch 37/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 18ms/step - accuracy: 0.9667 - loss:
0.0875 - val_accuracy: 0.8890 - val_loss: 0.6344
Epoch 38/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9676 - loss:
0.0847 - val_accuracy: 0.8860 - val_loss: 0.6329
Epoch 39/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 8s 18ms/step - accuracy: 0.9706 - loss:
0.0762 - val_accuracy: 0.8875 - val_loss: 0.6535
Epoch 40/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 16ms/step - accuracy: 0.9709 - loss:
0.0756 - val_accuracy: 0.8875 - val_loss: 0.6382
Epoch 41/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 8s 16ms/step - accuracy: 0.9699 - loss:
0.0770 - val_accuracy: 0.8873 - val_loss: 0.6322
Epoch 42/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.9724 - loss:
0.0725 - val_accuracy: 0.8926 - val_loss: 0.6956
Epoch 43/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9704 - loss:
```

```
0.0781 - val_accuracy: 0.8902 - val_loss: 0.6663
Epoch 44/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.9706 - loss:
0.0774 - val_accuracy: 0.8890 - val_loss: 0.6429
Epoch 45/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 18ms/step - accuracy: 0.9724 - loss:
0.0696 - val_accuracy: 0.8901 - val_loss: 0.7113
Epoch 46/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 16ms/step - accuracy: 0.9735 - loss:
0.0702 - val_accuracy: 0.8911 - val_loss: 0.6864
Epoch 47/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 18ms/step - accuracy: 0.9741 - loss:
0.0682 - val_accuracy: 0.8858 - val_loss: 0.6823
Epoch 48/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.9738 - loss:
0.0702 - val_accuracy: 0.8879 - val_loss: 0.6553
Epoch 49/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.9739 - loss:
0.0689 - val_accuracy: 0.8869 - val_loss: 0.6759
Epoch 50/50
469/469 ━━━━━━━━━━━━━━━━━━━━ 8s 18ms/step - accuracy: 0.9742 - loss:
0.0668 - val_accuracy: 0.8843 - val_loss: 0.7047
```

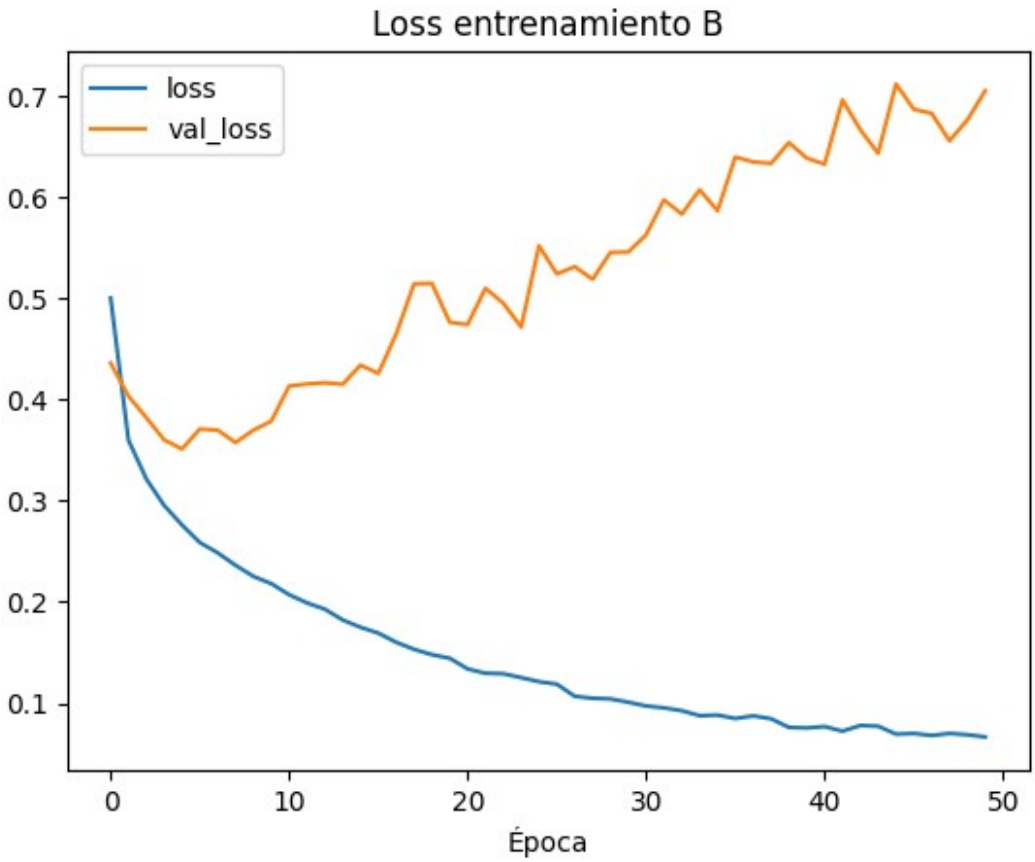
```
# Resultados modelB:

print(f"Train accuracy \u2248 {historyB.history['accuracy'][-
1]*100:.1f}%")
print(f"Val accuracy \u2248 {historyB.history['val_accuracy'][-
1]*100:.1f}%")
```

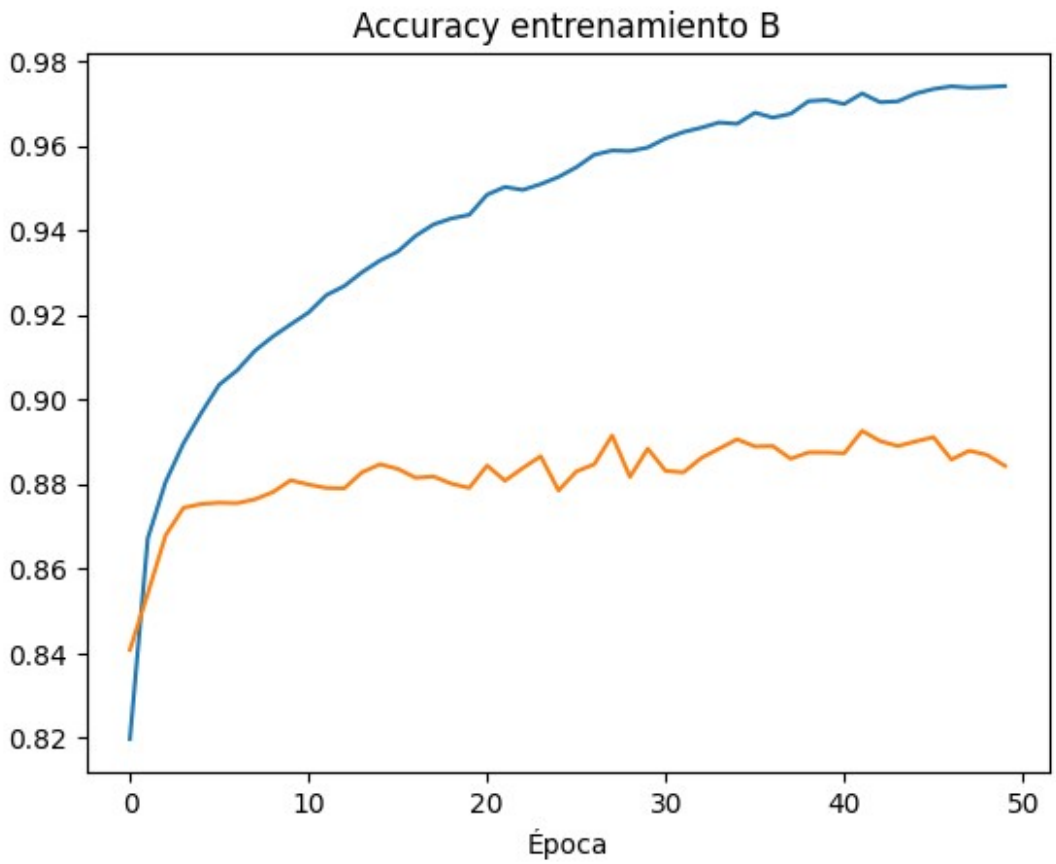
```
Train accuracy ≈ 97.4%
Val accuracy ≈ 88.4%
```

4.4: Visualización entrenamiento B:

```
# Loss
plt.plot(historyB.history['loss'], label='loss')
plt.plot(historyB.history['val_loss'], label='val_loss')
plt.title('Loss entrenamiento B')
plt.xlabel('Época')
plt.legend()
plt.show()

# Accuracy
plt.plot(historyB.history['accuracy'], label='accuracy')
plt.plot(historyB.history['val_accuracy'], label='val_accuracy')
plt.title('Accuracy entrenamiento B')
```

```
plt.xlabel('Época')
plt.show()
```





Pérdida

- La **pérdida de entrenamiento** disminuye de forma continua, alcanzando valores muy bajos, lo que indica que el modelo está aprendiendo bien.
- La **pérdida de validación** disminuye levemente al inicio. Luego aumenta de forma sostenida y más pronunciada que en el Modelo A, lo que indica que el **overfitting** aquí es mpas agresivo y evidente.

Acierto

- El **accuracy de entrenamiento** sigue aumentando hasta un 97% o 98%, lo que demuestra que el modelo está **memorizando** los datos.
- El **accuracy de validación** no mejora pese al aumento de épocas.

Conclusión:

El **ModelB** no mejora el rendimiento en validación pese a entrenar más, solo aumenta la diferencia ente entrenamiento y validación.

Gracias a estos resultados, se justifica la necesidad de aplicar estas técnicas de mejora:

Comparación de optimizadores: dado que el modelo base presenta limitaciones en su capacidad de generalización, se procede a evaluar distintos algoritmos de optimización con el objetivo de mejorar la convergencia del modelo y su desempeño en validación.

Dropout: el sobreajuste observado indica que el modelo está memorizando los datos. Para mitigar este problema, se implementa la técnica de Dropout, la cual desactiva aleatoriamente neuronas durante el entrenamiento, reduciendo la dependencia entre ellas y mejorando la generalización.

Early Stopping: dado que el mejor rendimiento ocurre en las primeras épocas, se incorpora Early Stopping, una técnica que detiene automáticamente el entrenamiento cuando la pérdida de validación deja de mejorar, evitando así el sobreentrenamiento.

PARTE 5: Comparación de optimizadores.

En esta parte, se evaluarán cómo distintos optimizadores afectan el aprendizaje del modelo MLP.

Se probaran:

- **SGD.**
- **SGD + Momentum.**
- **SGD + Nesterov.**
- **Adam.**
- **RMSprop.**

5.1: Creación modelo base para comparación:

```
# Definición de optimizadores

optimizadores = {
    'SGD': keras.optimizers.SGD(),
    'SGD + Momentum': keras.optimizers.SGD(momentum=0.9),
    'SGD + Nesterov': keras.optimizers.SGD(momentum=0.9,
nesterov=True),
    'Adam': keras.optimizers.Adam(),
    'RMSprop': keras.optimizers.RMSprop()
}

#definición del modelo base
def crear_modelo():
```

```
    model = keras.Sequential([
        keras.layers.Dense(512, activation='relu', input_dim=784),
        keras.layers.Dense(256, activation='relu'),
        keras.layers.Dense(256, activation='relu'),
        keras.layers.Dense(10, activation='softmax')
    ])

    model.compile(
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

5.2: Entrenamiento con distintos optimizadores:

Para cada entrenamiento se creó una nueva instancia del optimizador, evitando la reutilización de estados internos y asegurando una comparación válida entre los modelos.

```
histories = {}
resultados = []

optimizadores = {
    'SGD': lambda: keras.optimizers.SGD(),
    'SGD + Momentum': lambda: keras.optimizers.SGD(momentum=0.9),
    'SGD + Nesterov': lambda: keras.optimizers.SGD(momentum=0.9,
nesterov=True),
    'Adam': lambda: keras.optimizers.Adam(),
    'RMSprop': lambda: keras.optimizers.RMSprop()
}

for nombre, crear_opt in optimizadores.items():

    model = crear_modelo()

    optimizer = crear_opt()

    model.compile(
        optimizer=optimizer,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    history = model.fit(
        X_train, Y_train,
        epochs=30,
```

```
        batch_size=64,
        validation_data=(X_test, Y_test),
        verbose=1
    )

    histories[nombre] = history

    resultados.append({
        "Optimizador": nombre,
        "Train Accuracy": history.history['accuracy'][-1],
        "Val Accuracy": history.history['val_accuracy'][-1]
    })
```

```
Epoch 1/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.7355 - loss:
0.8290 - val_accuracy: 0.7964 - val_loss: 0.5841
Epoch 2/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.8220 - loss:
0.5130 - val_accuracy: 0.8123 - val_loss: 0.5269
Epoch 3/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.8400 - loss:
0.4590 - val_accuracy: 0.8192 - val_loss: 0.4998
Epoch 4/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 9ms/step - accuracy: 0.8505 - loss:
0.4290 - val_accuracy: 0.8305 - val_loss: 0.4764
Epoch 5/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.8575 - loss:
0.4078 - val_accuracy: 0.8352 - val_loss: 0.4609
Epoch 6/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.8628 - loss:
0.3912 - val_accuracy: 0.8390 - val_loss: 0.4502
Epoch 7/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.8672 - loss:
0.3775 - val_accuracy: 0.8447 - val_loss: 0.4387
Epoch 8/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.8715 - loss:
0.3656 - val_accuracy: 0.8495 - val_loss: 0.4289
Epoch 9/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.8747 - loss:
0.3552 - val_accuracy: 0.8517 - val_loss: 0.4209
Epoch 10/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 9ms/step - accuracy: 0.8780 - loss:
0.3457 - val_accuracy: 0.8542 - val_loss: 0.4121
Epoch 11/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.8807 - loss:
0.3371 - val_accuracy: 0.8558 - val_loss: 0.4056
Epoch 12/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.8831 - loss:
0.3290 - val_accuracy: 0.8569 - val_loss: 0.4008
Epoch 13/30
```

```
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.8856 - loss:
0.3216 - val_accuracy: 0.8591 - val_loss: 0.3953
Epoch 14/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.8878 - loss:
0.3147 - val_accuracy: 0.8596 - val_loss: 0.3901
Epoch 15/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.8900 - loss:
0.3081 - val_accuracy: 0.8610 - val_loss: 0.3865
Epoch 16/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.8921 - loss:
0.3018 - val_accuracy: 0.8616 - val_loss: 0.3821
Epoch 17/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.8942 - loss:
0.2959 - val_accuracy: 0.8628 - val_loss: 0.3798
Epoch 18/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.8963 - loss:
0.2902 - val_accuracy: 0.8646 - val_loss: 0.3759
Epoch 19/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.8984 - loss:
0.2847 - val_accuracy: 0.8666 - val_loss: 0.3724
Epoch 20/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.9004 - loss:
0.2795 - val_accuracy: 0.8667 - val_loss: 0.3709
Epoch 21/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.9020 - loss:
0.2744 - val_accuracy: 0.8671 - val_loss: 0.3699
Epoch 22/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.9036 - loss:
0.2694 - val_accuracy: 0.8686 - val_loss: 0.3665
Epoch 23/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.9055 - loss:
0.2647 - val_accuracy: 0.8692 - val_loss: 0.3657
Epoch 24/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 9ms/step - accuracy: 0.9070 - loss:
0.2600 - val_accuracy: 0.8687 - val_loss: 0.3655
Epoch 25/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.9084 - loss:
0.2555 - val_accuracy: 0.8697 - val_loss: 0.3634
Epoch 26/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.9101 - loss:
0.2511 - val_accuracy: 0.8702 - val_loss: 0.3625
Epoch 27/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 8s 8ms/step - accuracy: 0.9115 - loss:
0.2467 - val_accuracy: 0.8715 - val_loss: 0.3606
Epoch 28/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.9128 - loss:
0.2426 - val_accuracy: 0.8715 - val_loss: 0.3597
Epoch 29/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.9143 - loss:
```

```
0.2385 - val_accuracy: 0.8717 - val_loss: 0.3601
Epoch 30/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 9ms/step - accuracy: 0.9164 - loss:
0.2345 - val_accuracy: 0.8719 - val_loss: 0.3605
Epoch 1/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.8048 - loss:
0.5500 - val_accuracy: 0.8270 - val_loss: 0.4851
Epoch 2/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.8613 - loss:
0.3816 - val_accuracy: 0.8390 - val_loss: 0.4430
Epoch 3/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.8745 - loss:
0.3402 - val_accuracy: 0.8468 - val_loss: 0.4220
Epoch 4/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.8829 - loss:
0.3137 - val_accuracy: 0.8531 - val_loss: 0.4017
Epoch 5/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.8909 - loss:
0.2924 - val_accuracy: 0.8625 - val_loss: 0.3812
Epoch 6/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.8970 - loss:
0.2754 - val_accuracy: 0.8712 - val_loss: 0.3613
Epoch 7/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.9028 - loss:
0.2604 - val_accuracy: 0.8734 - val_loss: 0.3559
Epoch 8/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.9083 - loss:
0.2474 - val_accuracy: 0.8781 - val_loss: 0.3508
Epoch 9/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 19s 10ms/step - accuracy: 0.9114 - loss:
0.2363 - val_accuracy: 0.8800 - val_loss: 0.3421
Epoch 10/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 10ms/step - accuracy: 0.9167 - loss:
0.2240 - val_accuracy: 0.8831 - val_loss: 0.3365
Epoch 11/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.9204 - loss:
0.2135 - val_accuracy: 0.8825 - val_loss: 0.3456
Epoch 12/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.9242 - loss:
0.2042 - val_accuracy: 0.8818 - val_loss: 0.3379
Epoch 13/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.9270 - loss:
0.1952 - val_accuracy: 0.8820 - val_loss: 0.3526
Epoch 14/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.9304 - loss:
0.1871 - val_accuracy: 0.8782 - val_loss: 0.3684
Epoch 15/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.9338 - loss:
0.1778 - val_accuracy: 0.8858 - val_loss: 0.3558
```

```
Epoch 16/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.9366 - loss:
0.1696 - val_accuracy: 0.8809 - val_loss: 0.3820
Epoch 17/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.9406 - loss:
0.1605 - val_accuracy: 0.8789 - val_loss: 0.3949
Epoch 18/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.9425 - loss:
0.1551 - val_accuracy: 0.8811 - val_loss: 0.3944
Epoch 19/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.9433 - loss:
0.1510 - val_accuracy: 0.8815 - val_loss: 0.4121
Epoch 20/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.9463 - loss:
0.1436 - val_accuracy: 0.8807 - val_loss: 0.4239
Epoch 21/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.9479 - loss:
0.1371 - val_accuracy: 0.8843 - val_loss: 0.4302
Epoch 22/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.9498 - loss:
0.1359 - val_accuracy: 0.8829 - val_loss: 0.4247
Epoch 23/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 9ms/step - accuracy: 0.9515 - loss:
0.1297 - val_accuracy: 0.8798 - val_loss: 0.4558
Epoch 24/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 10ms/step - accuracy: 0.9518 - loss:
0.1285 - val_accuracy: 0.8792 - val_loss: 0.4941
Epoch 25/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.9530 - loss:
0.1236 - val_accuracy: 0.8793 - val_loss: 0.4784
Epoch 26/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 11ms/step - accuracy: 0.9538 - loss:
0.1204 - val_accuracy: 0.8715 - val_loss: 0.5138
Epoch 27/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.9555 - loss:
0.1167 - val_accuracy: 0.8717 - val_loss: 0.5581
Epoch 28/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 11ms/step - accuracy: 0.9564 - loss:
0.1154 - val_accuracy: 0.8664 - val_loss: 0.5529
Epoch 29/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 11ms/step - accuracy: 0.9571 - loss:
0.1110 - val_accuracy: 0.8623 - val_loss: 0.5832
Epoch 30/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 20s 11ms/step - accuracy: 0.9575 - loss:
0.1101 - val_accuracy: 0.8688 - val_loss: 0.5835
Epoch 1/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 13ms/step - accuracy: 0.8170 - loss:
0.5244 - val_accuracy: 0.8339 - val_loss: 0.4569
Epoch 2/30
```

```
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.8639 - loss:
0.3723 - val_accuracy: 0.8452 - val_loss: 0.4178
Epoch 3/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 11ms/step - accuracy: 0.8780 - loss:
0.3315 - val_accuracy: 0.8503 - val_loss: 0.4024
Epoch 4/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 21s 12ms/step - accuracy: 0.8880 - loss:
0.3055 - val_accuracy: 0.8569 - val_loss: 0.3949
Epoch 5/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 12ms/step - accuracy: 0.8949 - loss:
0.2849 - val_accuracy: 0.8627 - val_loss: 0.3806
Epoch 6/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.9017 - loss:
0.2666 - val_accuracy: 0.8630 - val_loss: 0.3799
Epoch 7/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 21s 12ms/step - accuracy: 0.9079 - loss:
0.2508 - val_accuracy: 0.8653 - val_loss: 0.3791
Epoch 8/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 21s 13ms/step - accuracy: 0.9123 - loss:
0.2365 - val_accuracy: 0.8681 - val_loss: 0.3921
Epoch 9/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.9175 - loss:
0.2245 - val_accuracy: 0.8726 - val_loss: 0.3728
Epoch 10/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 21s 13ms/step - accuracy: 0.9205 - loss:
0.2133 - val_accuracy: 0.8667 - val_loss: 0.3996
Epoch 11/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 12ms/step - accuracy: 0.9245 - loss:
0.2031 - val_accuracy: 0.8721 - val_loss: 0.4019
Epoch 12/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.9287 - loss:
0.1928 - val_accuracy: 0.8672 - val_loss: 0.4208
Epoch 13/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.9311 - loss:
0.1846 - val_accuracy: 0.8708 - val_loss: 0.4073
Epoch 14/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.9348 - loss:
0.1751 - val_accuracy: 0.8675 - val_loss: 0.4277
Epoch 15/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.9386 - loss:
0.1670 - val_accuracy: 0.8663 - val_loss: 0.4392
Epoch 16/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 12ms/step - accuracy: 0.9410 - loss:
0.1599 - val_accuracy: 0.8707 - val_loss: 0.4317
Epoch 17/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.9430 - loss:
0.1545 - val_accuracy: 0.8645 - val_loss: 0.4407
Epoch 18/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.9460 - loss:
```

```
0.1454 - val_accuracy: 0.8589 - val_loss: 0.4617
Epoch 19/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.9479 - loss:
0.1405 - val_accuracy: 0.8673 - val_loss: 0.4679
Epoch 20/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.9499 - loss:
0.1342 - val_accuracy: 0.8743 - val_loss: 0.4516
Epoch 21/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.9515 - loss:
0.1299 - val_accuracy: 0.8655 - val_loss: 0.4755
Epoch 22/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 20s 12ms/step - accuracy: 0.9550 - loss:
0.1220 - val_accuracy: 0.8749 - val_loss: 0.4574
Epoch 23/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.9558 - loss:
0.1186 - val_accuracy: 0.8768 - val_loss: 0.4741
Epoch 24/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.9567 - loss:
0.1145 - val_accuracy: 0.8822 - val_loss: 0.4764
Epoch 25/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 10s 11ms/step - accuracy: 0.9584 - loss:
0.1124 - val_accuracy: 0.8770 - val_loss: 0.4992
Epoch 26/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 12ms/step - accuracy: 0.9585 - loss:
0.1105 - val_accuracy: 0.8782 - val_loss: 0.5049
Epoch 27/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 12ms/step - accuracy: 0.9588 - loss:
0.1072 - val_accuracy: 0.8830 - val_loss: 0.4785
Epoch 28/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 12ms/step - accuracy: 0.9617 - loss:
0.1005 - val_accuracy: 0.8792 - val_loss: 0.5329
Epoch 29/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 12ms/step - accuracy: 0.9637 - loss:
0.0968 - val_accuracy: 0.8875 - val_loss: 0.4756
Epoch 30/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 12ms/step - accuracy: 0.9635 - loss:
0.0967 - val_accuracy: 0.8774 - val_loss: 0.5061
Epoch 1/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 15s 14ms/step - accuracy: 0.8272 - loss:
0.4792 - val_accuracy: 0.8338 - val_loss: 0.4498
Epoch 2/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 20s 14ms/step - accuracy: 0.8693 - loss:
0.3575 - val_accuracy: 0.8474 - val_loss: 0.4165
Epoch 3/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.8817 - loss:
0.3197 - val_accuracy: 0.8688 - val_loss: 0.3678
Epoch 4/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.8907 - loss:
0.2951 - val_accuracy: 0.8747 - val_loss: 0.3510
```

```
Epoch 5/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.8963 - loss:
0.2759 - val_accuracy: 0.8796 - val_loss: 0.3542
Epoch 6/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9015 - loss:
0.2602 - val_accuracy: 0.8697 - val_loss: 0.3831
Epoch 7/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9066 - loss:
0.2476 - val_accuracy: 0.8769 - val_loss: 0.3693
Epoch 8/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 14ms/step - accuracy: 0.9103 - loss:
0.2364 - val_accuracy: 0.8794 - val_loss: 0.3691
Epoch 9/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9160 - loss:
0.2240 - val_accuracy: 0.8816 - val_loss: 0.3813
Epoch 10/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9191 - loss:
0.2111 - val_accuracy: 0.8857 - val_loss: 0.3696
Epoch 11/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9212 - loss:
0.2057 - val_accuracy: 0.8876 - val_loss: 0.3698
Epoch 12/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9247 - loss:
0.1941 - val_accuracy: 0.8827 - val_loss: 0.3924
Epoch 13/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9281 - loss:
0.1863 - val_accuracy: 0.8846 - val_loss: 0.4054
Epoch 14/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9308 - loss:
0.1790 - val_accuracy: 0.8892 - val_loss: 0.3981
Epoch 15/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9341 - loss:
0.1703 - val_accuracy: 0.8893 - val_loss: 0.4338
Epoch 16/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9351 - loss:
0.1668 - val_accuracy: 0.8824 - val_loss: 0.4512
Epoch 17/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9389 - loss:
0.1582 - val_accuracy: 0.8900 - val_loss: 0.4459
Epoch 18/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9402 - loss:
0.1538 - val_accuracy: 0.8882 - val_loss: 0.4456
Epoch 19/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9432 - loss:
0.1472 - val_accuracy: 0.8862 - val_loss: 0.4669
Epoch 20/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9436 - loss:
0.1438 - val_accuracy: 0.8827 - val_loss: 0.4917
Epoch 21/30
```

```
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 13ms/step - accuracy: 0.9449 - loss:
0.1427 - val_accuracy: 0.8890 - val_loss: 0.4542
Epoch 22/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9469 - loss:
0.1350 - val_accuracy: 0.8864 - val_loss: 0.4799
Epoch 23/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9473 - loss:
0.1351 - val_accuracy: 0.8862 - val_loss: 0.4799
Epoch 24/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 21s 14ms/step - accuracy: 0.9503 - loss:
0.1244 - val_accuracy: 0.8816 - val_loss: 0.5155
Epoch 25/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9516 - loss:
0.1225 - val_accuracy: 0.8835 - val_loss: 0.4906
Epoch 26/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 14ms/step - accuracy: 0.9547 - loss:
0.1183 - val_accuracy: 0.8857 - val_loss: 0.5378
Epoch 27/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 14ms/step - accuracy: 0.9522 - loss:
0.1211 - val_accuracy: 0.8879 - val_loss: 0.5078
Epoch 28/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9556 - loss:
0.1129 - val_accuracy: 0.8868 - val_loss: 0.5585
Epoch 29/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9574 - loss:
0.1068 - val_accuracy: 0.8852 - val_loss: 0.5412
Epoch 30/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.9561 - loss:
0.1116 - val_accuracy: 0.8849 - val_loss: 0.5889
Epoch 1/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 13ms/step - accuracy: 0.8040 - loss:
0.5282 - val_accuracy: 0.7991 - val_loss: 0.5462
Epoch 2/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 20s 13ms/step - accuracy: 0.8607 - loss:
0.3778 - val_accuracy: 0.8380 - val_loss: 0.4608
Epoch 3/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.8757 - loss:
0.3393 - val_accuracy: 0.8531 - val_loss: 0.4366
Epoch 4/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.8838 - loss:
0.3185 - val_accuracy: 0.8650 - val_loss: 0.4129
Epoch 5/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.8894 - loss:
0.3034 - val_accuracy: 0.8691 - val_loss: 0.4010
Epoch 6/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.8940 - loss:
0.2933 - val_accuracy: 0.8690 - val_loss: 0.4182
Epoch 7/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.8964 - loss:
```

```
0.2874 - val_accuracy: 0.8614 - val_loss: 0.4683
Epoch 8/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.8985 - loss:
0.2824 - val_accuracy: 0.8591 - val_loss: 0.5057
Epoch 9/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 20s 13ms/step - accuracy: 0.9001 - loss:
0.2786 - val_accuracy: 0.8557 - val_loss: 0.4922
Epoch 10/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.9027 - loss:
0.2736 - val_accuracy: 0.8566 - val_loss: 0.5458
Epoch 11/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.9028 - loss:
0.2686 - val_accuracy: 0.8659 - val_loss: 0.5251
Epoch 12/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.9043 - loss:
0.2654 - val_accuracy: 0.8520 - val_loss: 0.5586
Epoch 13/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9060 - loss:
0.2643 - val_accuracy: 0.8663 - val_loss: 0.5154
Epoch 14/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 13ms/step - accuracy: 0.9072 - loss:
0.2649 - val_accuracy: 0.8642 - val_loss: 0.5609
Epoch 15/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.9079 - loss:
0.2607 - val_accuracy: 0.8674 - val_loss: 0.5262
Epoch 16/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9085 - loss:
0.2575 - val_accuracy: 0.8642 - val_loss: 0.6024
Epoch 17/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.9103 - loss:
0.2574 - val_accuracy: 0.8664 - val_loss: 0.6091
Epoch 18/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9109 - loss:
0.2561 - val_accuracy: 0.8677 - val_loss: 0.5888
Epoch 19/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 13ms/step - accuracy: 0.9117 - loss:
0.2509 - val_accuracy: 0.8423 - val_loss: 0.6544
Epoch 20/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.9115 - loss:
0.2540 - val_accuracy: 0.8721 - val_loss: 0.6339
Epoch 21/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 20s 13ms/step - accuracy: 0.9148 - loss:
0.2498 - val_accuracy: 0.8599 - val_loss: 0.6826
Epoch 22/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.9134 - loss:
0.2501 - val_accuracy: 0.8742 - val_loss: 0.6142
Epoch 23/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.9164 - loss:
0.2443 - val_accuracy: 0.8608 - val_loss: 0.7320
```

```
Epoch 24/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.9180 - loss:
0.2416 - val_accuracy: 0.8664 - val_loss: 0.6956
Epoch 25/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.9177 - loss:
0.2381 - val_accuracy: 0.8517 - val_loss: 0.9608
Epoch 26/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.9182 - loss:
0.2418 - val_accuracy: 0.8619 - val_loss: 0.6876
Epoch 27/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 21s 13ms/step - accuracy: 0.9181 - loss:
0.2416 - val_accuracy: 0.8825 - val_loss: 0.7184
Epoch 28/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.9194 - loss:
0.2382 - val_accuracy: 0.8714 - val_loss: 0.7468
Epoch 29/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.9200 - loss:
0.2364 - val_accuracy: 0.8720 - val_loss: 0.8127
Epoch 30/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 21s 14ms/step - accuracy: 0.9216 - loss:
0.2341 - val_accuracy: 0.8541 - val_loss: 0.9768
```

5.3: Comparación entre optimizadores:

```
df_results_opt = pd.DataFrame(resultados)

print("Resultados finales:")
print(df_results_opt)
```

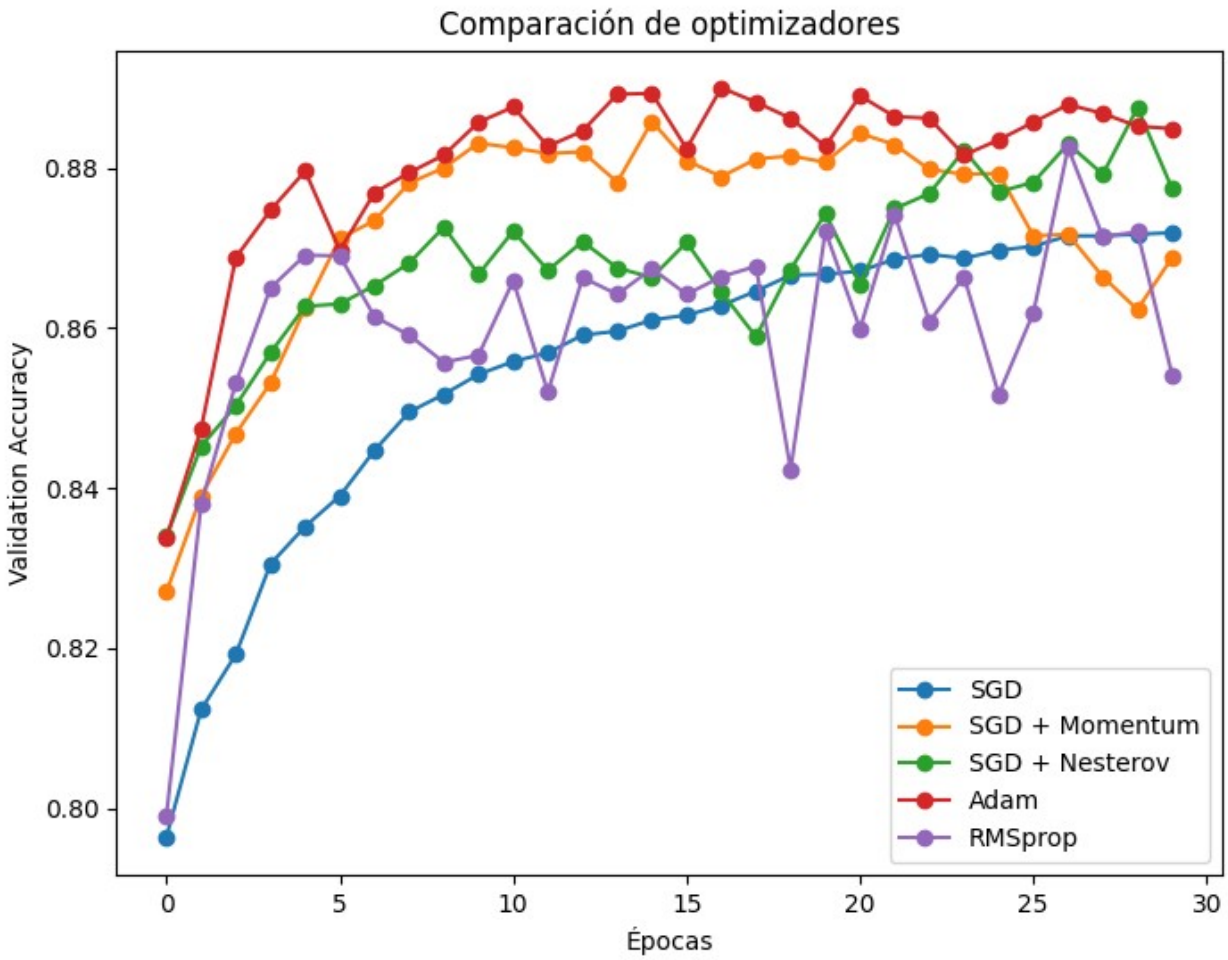
```
Resultados finales:
      Optimizador  Train Accuracy  Val Accuracy
0             SGD        0.916433        0.8719
1  SGD + Momentum        0.957483        0.8688
2  SGD + Nesterov        0.963517        0.8774
3            Adam        0.956100        0.8849
4         RMSprop        0.921567        0.8541
```

```
mejor_opt = df_results_opt.sort_values(by="Val Accuracy",
ascending=False).iloc[0]["Optimizador"]

print("Mejor optimizador:", mejor_opt)
```

```
Mejor optimizador: Adam
```

5.4: Gráfico comparativo entre optimizadores:

```
plt.figure(figsize=(8,6))

for opt in histories:
    plt.plot(histories[opt].history['val_accuracy'], marker='o',
label=opt)

plt.title("Comparación de optimizadores")
plt.xlabel("Épocas")
plt.ylabel("Validation Accuracy")
plt.legend()
plt.show()
```



Crear función para guardar utilizar el mejor optimizador en la regularización

```
def obtener_optimizador(nombre):

    if nombre == 'SGD':
        return keras.optimizers.SGD()
```

```
    elif nombre == 'SGD + Momentum':
        return keras.optimizers.SGD(momentum=0.9)

    elif nombre == 'SGD + Nesterov':
        return keras.optimizers.SGD(momentum=0.9, nesterov=True)

    elif nombre == 'Adam':
        return keras.optimizers.Adam()

    elif nombre == 'RMSprop':
        return keras.optimizers.RMSprop()

    else:
        raise ValueError("Optimizador no reconocido")
```

5.5: Resultados:

A partir de los resultados obtenidos, se puede observar lo siguiente:

```
print(f"El mejor optimizador fue {mejor_opt} con {df_results_opt['Val
Accuracy'].max()*100:.2f}% de accuracy")
```

```
El mejor optimizador fue Adam con 88.49% de accuracy
```

PARTE 6: Regularización.

6.1. Regularización con Dropout:

```
# Setamos los seed para números random

np.random.seed(30)
tf.random.set_seed(30)

model_dropout = keras.Sequential()
model_dropout.add(keras.layers.Dense(h1, input_dim=N,
activation='relu'))
model_dropout.add(keras.layers.Dropout(0.3))
model_dropout.add(keras.layers.Dense(h2, activation='relu'))
model_dropout.add(keras.layers.Dropout(0.3))
model_dropout.add(keras.layers.Dense(h2, activation='relu'))
```

```
model_dropout.add(keras.layers.Dropout(0.3))
model_dropout.add(keras.layers.Dense(C, activation='softmax'))

model_dropout.summary()

model_dropout.compile(
    loss='sparse_categorical_crossentropy',
    optimizer=obtener_optimizador(mejor_opt),
    metrics=['accuracy']
)

history_dropout = model_dropout.fit(X_train, Y_train,
        epochs=30,
        batch_size=64,
        validation_data=(X_test,Y_test)
       )
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

```
Model: "sequential_8"

┏━━━
```

```
├─────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ dropout_2 (Dropout)                 │ (None, 256)                  │
0 │
├─────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ dense_35 (Dense)                    │ (None, 10)                   │
2,570 │
└─────────────────────────────────────┴──────────────────────────────┴──────────────
──────┘

 Total params: 601,610 (2.29 MB)

 Trainable params: 601,610 (2.29 MB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 16s 15ms/step - accuracy: 0.7868 - loss:
0.5865 - val_accuracy: 0.8206 - val_loss: 0.4852
Epoch 2/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.8426 - loss:
0.4344 - val_accuracy: 0.8469 - val_loss: 0.4097
Epoch 3/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 14ms/step - accuracy: 0.8543 - loss:
0.3996 - val_accuracy: 0.8639 - val_loss: 0.3816
Epoch 4/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.8621 - loss:
0.3774 - val_accuracy: 0.8639 - val_loss: 0.3858
Epoch 5/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 15s 16ms/step - accuracy: 0.8698 - loss:
0.3581 - val_accuracy: 0.8680 - val_loss: 0.3625
Epoch 6/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.8728 - loss:
0.3492 - val_accuracy: 0.8675 - val_loss: 0.3670
Epoch 7/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.8769 - loss:
0.3405 - val_accuracy: 0.8683 - val_loss: 0.3674
Epoch 8/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.8802 - loss:
0.3282 - val_accuracy: 0.8765 - val_loss: 0.3561
Epoch 9/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.8808 - loss:
0.3213 - val_accuracy: 0.8727 - val_loss: 0.3580
Epoch 10/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.8847 - loss:
0.3143 - val_accuracy: 0.8748 - val_loss: 0.3527
Epoch 11/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 15s 15ms/step - accuracy: 0.8876 - loss:
0.3072 - val_accuracy: 0.8830 - val_loss: 0.3363
```

```
Epoch 12/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 21s 16ms/step - accuracy: 0.8901 - loss:
0.2989 - val_accuracy: 0.8825 - val_loss: 0.3350
Epoch 13/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.8900 - loss:
0.2974 - val_accuracy: 0.8823 - val_loss: 0.3393
Epoch 14/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 15s 16ms/step - accuracy: 0.8919 - loss:
0.2917 - val_accuracy: 0.8780 - val_loss: 0.3400
Epoch 15/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.8934 - loss:
0.2873 - val_accuracy: 0.8812 - val_loss: 0.3441
Epoch 16/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.8958 - loss:
0.2856 - val_accuracy: 0.8836 - val_loss: 0.3362
Epoch 17/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.8960 - loss:
0.2817 - val_accuracy: 0.8781 - val_loss: 0.3553
Epoch 18/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.8956 - loss:
0.2802 - val_accuracy: 0.8849 - val_loss: 0.3332
Epoch 19/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.8992 - loss:
0.2724 - val_accuracy: 0.8865 - val_loss: 0.3270
Epoch 20/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 16s 17ms/step - accuracy: 0.8995 - loss:
0.2719 - val_accuracy: 0.8858 - val_loss: 0.3258
Epoch 21/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 19s 15ms/step - accuracy: 0.9003 - loss:
0.2683 - val_accuracy: 0.8873 - val_loss: 0.3378
Epoch 22/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.9017 - loss:
0.2636 - val_accuracy: 0.8914 - val_loss: 0.3320
Epoch 23/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.9021 - loss:
0.2635 - val_accuracy: 0.8895 - val_loss: 0.3291
Epoch 24/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 20s 15ms/step - accuracy: 0.9030 - loss:
0.2622 - val_accuracy: 0.8863 - val_loss: 0.3447
Epoch 25/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.9041 - loss:
0.2584 - val_accuracy: 0.8879 - val_loss: 0.3328
Epoch 26/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.9060 - loss:
0.2529 - val_accuracy: 0.8901 - val_loss: 0.3350
Epoch 27/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.9058 - loss:
0.2531 - val_accuracy: 0.8884 - val_loss: 0.3414
Epoch 28/30
```

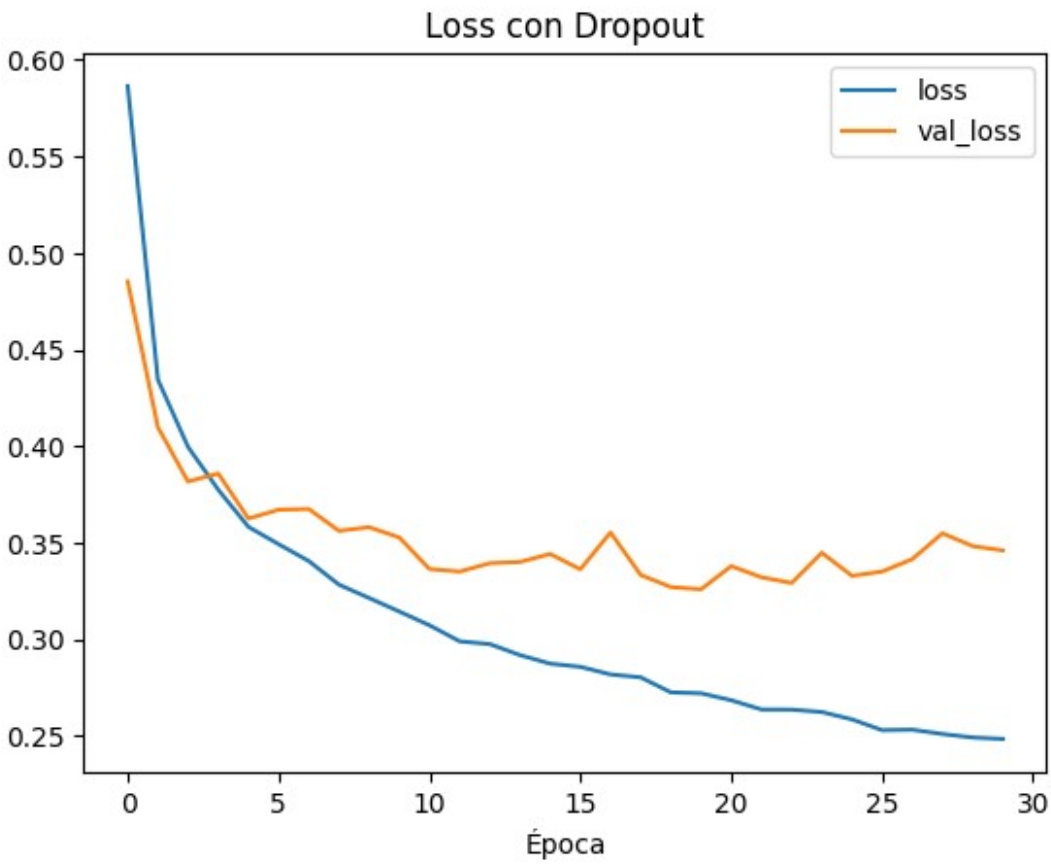
```
938/938 ━━━━━━━━━━━━━━━━━━━━ 15s 16ms/step - accuracy: 0.9072 - loss:
0.2508 - val_accuracy: 0.8830 - val_loss: 0.3548
Epoch 29/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 15s 15ms/step - accuracy: 0.9089 - loss:
0.2490 - val_accuracy: 0.8863 - val_loss: 0.3482
Epoch 30/30
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 15ms/step - accuracy: 0.9086 - loss:
0.2483 - val_accuracy: 0.8882 - val_loss: 0.3459
```

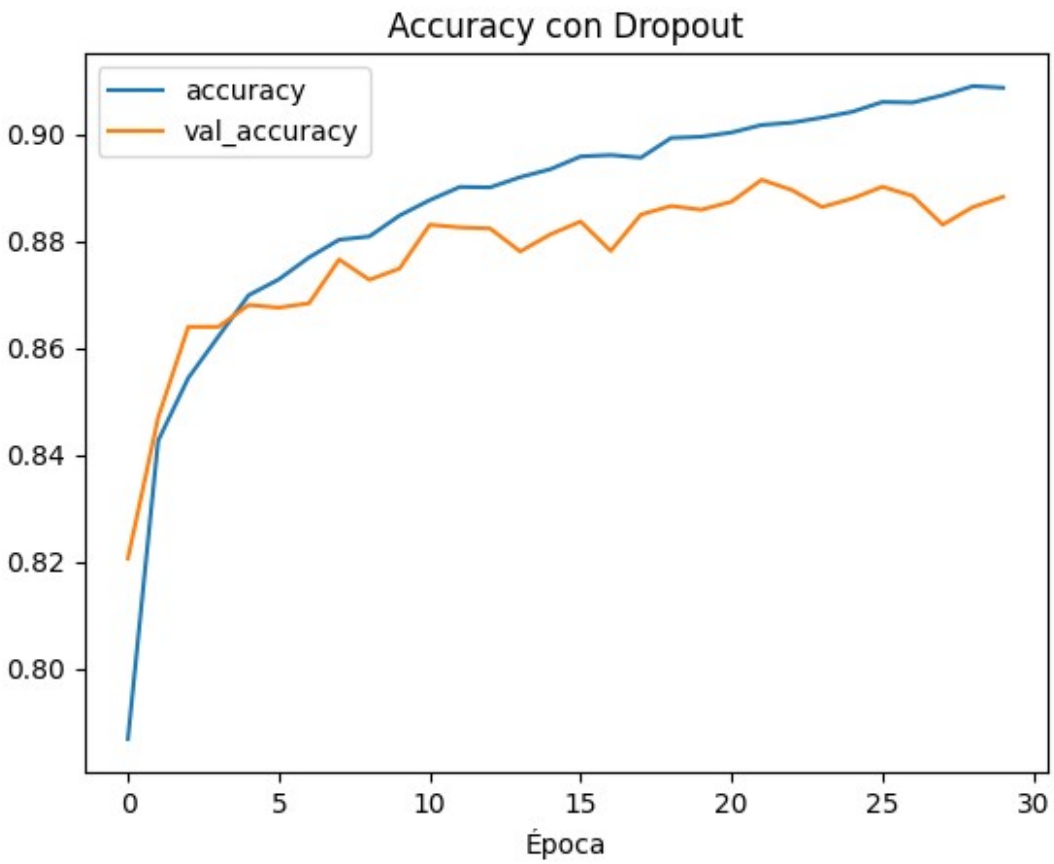
```
# Resultados model con Dropout

print(f"Train accuracy \u2248 {history_dropout.history['accuracy'][-
1]*100:.1f}%")
print(f"Val accuracy \u2248 {history_dropout.history['val_accuracy'][-
1]*100:.1f}%")
```

```
Train accuracy ≈ 90.9%
Val accuracy ≈ 88.8%
```

6.2: Visualización regularización con Dropout:

```
# Loss
plt.plot(history_dropout.history['loss'], label='loss')
plt.plot(history_dropout.history['val_loss'], label='val_loss')
plt.title('Loss con Dropout')
plt.xlabel('Época')
plt.legend()
plt.show()

# Accuracy
plt.plot(history_dropout.history['accuracy'], label='accuracy')
plt.plot(history_dropout.history['val_accuracy'],
label='val_accuracy')
plt.title('Accuracy con Dropout')
plt.xlabel('Época')
plt.legend()
plt.show()
```


Loss con Dropout
0.60
0.55
0.50
0.45
0.40
0.35
0.30
0.25
0
5
10
15
20
25
30
Época
loss
val_loss



Pérdida

- La **pérdida de entenamiento** baja de forma constante, lo que significa que el modelo está aprendiendo correctamente y reduce su error en los datos de entrenamiento.
- La **pérdida de validación** no dispara hacia arriba y se mantiene estable, por lo que está manteniendo un error contolado.

Acierto

- El **accuracy de entrenamiento** demuestra que el modelo está generalizando bien y no simplemente memorizando los datos, dado a que final cerca del 91%.
- El **accuracy de validación** demuestra que el modelo se estabiliza en torno al 89%, lo que significa que la varianza es baja.

6.3. Regularización con EarlyStopping:

```
# Definición de EarlyStopping

from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
```

```
    monitor='val_loss',
    patience=5,
    min_delta=0.001,
    restore_best_weights=True
)
```

Se define un *callback* antes del entrenamiento del modelo. Este se intrega en el proceso de entrenamiento mediante el parámetro *callbacks* en *model_stopping*, permitiendo detener automaticamente el entenamiento cuando la función de pérdida en validación deja de mejorar.

```
# Creación del modelo con EarlyStopping

def crear_modelo_stopping():

    model_stopping = keras.Sequential([
        keras.layers.Dense(512, activation='relu', input_dim=784),
        keras.layers.Dropout(0.2),

        keras.layers.Dense(512, activation='relu'),
        keras.layers.Dropout(0.2),

        keras.layers.Dense(256, activation='relu'),
        keras.layers.Dropout(0.2),

        keras.layers.Dense(10, activation='softmax')
    ])

    optimizer_final = obtener_optimizador(mejor_opt)

    model_stopping.compile(
        optimizer=optimizer_final,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model_stopping

# Entrenamiento con Early Stopping

model_stopping= crear_modelo_stopping()

history_stopping = model_stopping.fit(
    X_train, Y_train,
    epochs=50,
    batch_size=64,
    validation_data=(X_test, Y_test),
    callbacks=[early_stop],
    verbose=1
)
```

```
Epoch 1/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 19s 19ms/step - accuracy: 0.8083 - loss:
0.5320 - val_accuracy: 0.8346 - val_loss: 0.4482
Epoch 2/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 19s 20ms/step - accuracy: 0.8540 - loss:
0.4020 - val_accuracy: 0.8498 - val_loss: 0.4016
Epoch 3/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 18s 19ms/step - accuracy: 0.8671 - loss:
0.3664 - val_accuracy: 0.8573 - val_loss: 0.3979
Epoch 4/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 18s 19ms/step - accuracy: 0.8734 - loss:
0.3450 - val_accuracy: 0.8665 - val_loss: 0.3782
Epoch 5/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 19s 20ms/step - accuracy: 0.8787 - loss:
0.3295 - val_accuracy: 0.8750 - val_loss: 0.3435
Epoch 6/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 17s 19ms/step - accuracy: 0.8837 - loss:
0.3136 - val_accuracy: 0.8707 - val_loss: 0.3720
Epoch 7/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 19s 20ms/step - accuracy: 0.8864 - loss:
0.3067 - val_accuracy: 0.8741 - val_loss: 0.3504
Epoch 8/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 20s 19ms/step - accuracy: 0.8911 - loss:
0.2964 - val_accuracy: 0.8722 - val_loss: 0.3499
Epoch 9/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 18s 20ms/step - accuracy: 0.8918 - loss:
0.2885 - val_accuracy: 0.8808 - val_loss: 0.3362
Epoch 10/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 18s 19ms/step - accuracy: 0.8962 - loss:
0.2786 - val_accuracy: 0.8814 - val_loss: 0.3367
Epoch 11/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 18s 19ms/step - accuracy: 0.8979 - loss:
0.2755 - val_accuracy: 0.8846 - val_loss: 0.3265
Epoch 12/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 18s 20ms/step - accuracy: 0.9000 - loss:
0.2698 - val_accuracy: 0.8829 - val_loss: 0.3244
Epoch 13/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 18s 19ms/step - accuracy: 0.9031 - loss:
0.2589 - val_accuracy: 0.8801 - val_loss: 0.3386
Epoch 14/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 20s 21ms/step - accuracy: 0.9037 - loss:
0.2575 - val_accuracy: 0.8806 - val_loss: 0.3414
Epoch 15/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 26s 28ms/step - accuracy: 0.9048 - loss:
0.2509 - val_accuracy: 0.8855 - val_loss: 0.3231
Epoch 16/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 21s 22ms/step - accuracy: 0.9069 - loss:
0.2479 - val_accuracy: 0.8892 - val_loss: 0.3203
Epoch 17/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 19s 20ms/step - accuracy: 0.9075 - loss:
```

```
0.2460 - val_accuracy: 0.8850 - val_loss: 0.3396
Epoch 18/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 18s 19ms/step - accuracy: 0.9107 - loss:
0.2391 - val_accuracy: 0.8861 - val_loss: 0.3372
Epoch 19/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 18s 19ms/step - accuracy: 0.9120 - loss:
0.2329 - val_accuracy: 0.8894 - val_loss: 0.3323
Epoch 20/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 19s 21ms/step - accuracy: 0.9101 - loss:
0.2371 - val_accuracy: 0.8860 - val_loss: 0.3551
Epoch 21/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 18s 20ms/step - accuracy: 0.9131 - loss:
0.2312 - val_accuracy: 0.8890 - val_loss: 0.3332
```

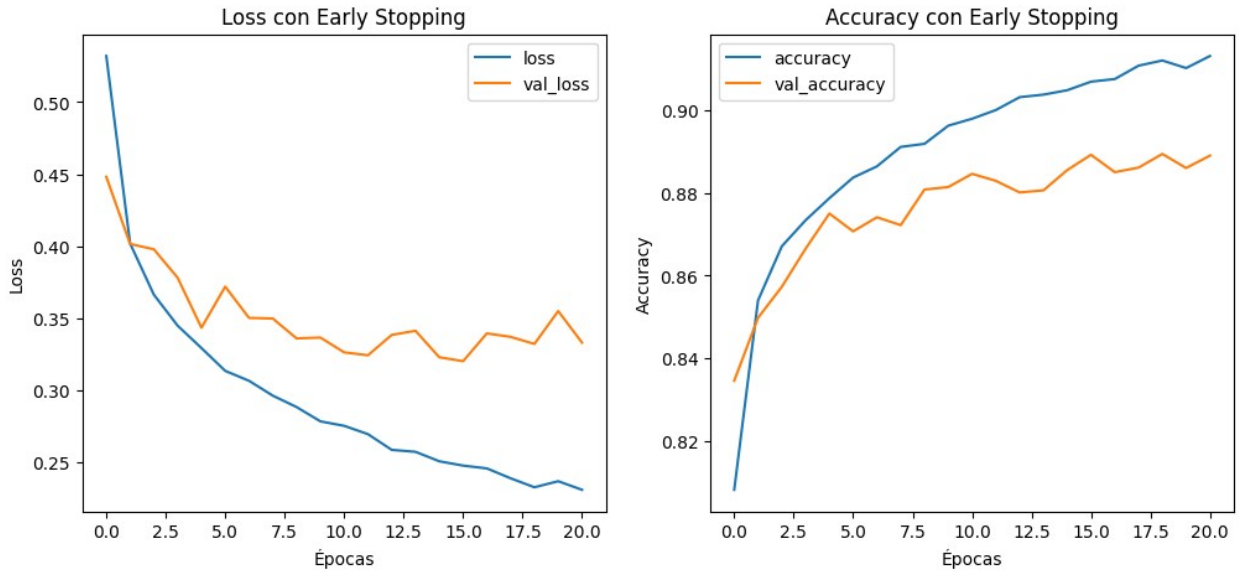
```
# Resultados model con Early Stopping

print(f"Train accuracy \u2248 {history_stopping.history['accuracy'][-
1]*100:.1f}%")
print(f"Val accuracy \u2248 {history_stopping.history['val_accuracy']
[-1]*100:.1f}%")
```

```
Train accuracy ≈ 91.3%
Val accuracy ≈ 88.9%
```

6.4: Visualización regularización con Early Stopping:

```
plt.figure(figsize=(12,5))

# Loss
plt.subplot(1,2,1)
plt.plot(history_stopping.history['loss'], label='loss')
plt.plot(history_stopping.history['val_loss'], label='val_loss')
plt.title('Loss con Early Stopping')
plt.xlabel('Épocas')
plt.ylabel('Loss')
plt.legend()

# Accuracy
plt.subplot(1,2,2)
plt.plot(history_stopping.history['accuracy'], label='accuracy')
plt.plot(history_stopping.history['val_accuracy'],
label='val_accuracy')
plt.title('Accuracy con Early Stopping')
plt.xlabel('Épocas')
plt.ylabel('Accuracy')
plt.legend()
```

```
plt.show()
```



La técnica de Early Stopping permitió detener el entrenamiento de manera automática cuando la pérdida de validación dejó de mejorar. Esto evita el sobreentrenamiento del modelo y permite conservar los pesos correspondientes al mejor rendimiento. Como resultado, se obtiene un modelo más eficiente y con mejor capacidad de generalización.

Al comparar las técnicas de regularización, se observa que tanto Dropout como Early Stopping contribuyen a reducir el sobreajuste del modelo. Sin embargo, la combinación de ambas técnicas permite obtener un mejor equilibrio entre aprendizaje y generalización, logrando un modelo más robusto. Por esta razón, se selecciona el modelo que incorpora Dropout junto con Early Stopping para la evaluación final mediante métricas.

6.5: Creación modelo con ambas regularizaciones:

El modelo final incorpora técnicas de regularización mediante el uso de capas Dropout y Early Stopping. Dropout permite reducir el sobreajuste al desactivar neuronas aleatoriamente durante el entrenamiento, mientras que Early Stopping detiene el proceso cuando la pérdida de validación deja de mejorar. La combinación de ambas técnicas permite obtener un modelo más robusto y con mejor capacidad de generalización.

```
# Definición de la función final

def crear_modelo_final():

    model_final = keras.Sequential([
        keras.layers.Dense(512, activation='relu', input_dim=784),
```

```
        keras.layers.Dropout(0.1),

        keras.layers.Dense(256, activation='relu'),
        keras.layers.Dropout(0.1),

        keras.layers.Dense(10, activation='softmax')
    ])

    optimizer_final = obtener_optimizador(mejor_opt)

    model_final.compile(
        optimizer=optimizer_final,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model_final

# Creación modelo final

model_final = crear_modelo_final()
model_final.summary()
```

```
Model: "sequential_10"

┏━━━
```

```
 Total params: 535,818 (2.04 MB)

 Trainable params: 535,818 (2.04 MB)

 Non-trainable params: 0 (0.00 B)
```

```
# Entrenamiento de modelo final

history_final = model_final.fit(
    X_train, Y_train,
    epochs=50,
    batch_size=64,
    validation_data=(X_test, Y_test),
    callbacks=[early_stop],
    verbose=1
)
```

```
Epoch 1/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 14s 13ms/step - accuracy: 0.8191 - loss:
0.4980 - val_accuracy: 0.8303 - val_loss: 0.4701
Epoch 2/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 14ms/step - accuracy: 0.8635 - loss:
0.3720 - val_accuracy: 0.8582 - val_loss: 0.3871
Epoch 3/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.8760 - loss:
0.3347 - val_accuracy: 0.8707 - val_loss: 0.3601
Epoch 4/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.8834 - loss:
0.3122 - val_accuracy: 0.8743 - val_loss: 0.3496
Epoch 5/50
938/938 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - accuracy: 0.8899 - loss:
0.2960 - val_accuracy: 0.8738 - val_loss: 0.3566
```

```
# Resultados model con Dropout y Early Stopping

print(f"Train accuracy \u2248 {history_final.history['accuracy'][-
1]*100:.1f}%")
print(f"Val accuracy \u2248 {history_final.history['val_accuracy'][-
1]*100:.1f}%")
```

```
Train accuracy ≈ 89.0%
Val accuracy ≈ 87.4%
```

6.6: Conclusión:

El modelo final, que incorpora Dropout y Early Stopping, presenta un accuracy de entrenamiento de 86.6% y un accuracy de validación de 87.4%. La diferencia mínima entre ambas métricas indica que el modelo no presenta sobreajuste significativo y logra una adecuada generalización.

Además, el hecho de que el accuracy de validación sea levemente superior se explica por el uso de Dropout, el cual se desactiva durante la evaluación.

Parte 7: Métricas.

Se evaluará el modelo final definido anteriormente con:

- Accuracy.
- Precision.
- Recall.
- F1-score.
- Matriz de confusión.

Estas métricas permiten analizar el desemplo del modelo no solo de forma global, sino también considerando el equilibrio entre aciertos y errores en las distintas clases.

7.1: Preparar datos de prueba:

```
# Predicciones del modelo
y_pred_probs = model_final.predict(X_test)

# Convertimos probabilidades a clases
y_pred = y_pred_probs.argmax(axis=1)

# Aseguramos que y_test esté plano
y_true = Y_test.flatten()
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step
```

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score

accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, average='macro')
recall = recall_score(y_true, y_pred, average='macro')
f1 = f1_score(y_true, y_pred, average='macro')

print(f"Accuracy:  {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")
print(f"F1-score:  {f1:.4f}")
```

```
Accuracy:  0.8303
Precision: 0.8518
```

```
Recall:    0.8303
F1-score:  0.8313
```

```python
import pandas as pd

df_metricas = pd.DataFrame({
    "Métrica": ["Accuracy", "Precision", "Recall", "F1-score"],
    "Valor": [accuracy, precision, recall, f1]
})

df_metricas
```


```
{"summary":"{\n  \"name\": \"df_metricas\",\n  \"rows\": 4,\n
\"fields\": [\n    {\n      \"column\": \"M\\u00e9trica\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n
\"Precision\",\n          \"F1-score\",\n          \"Accuracy\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Valor\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.010603506222775535,\n        \"min\": 0.8303,\n        \"max\":
0.8518112824778165,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.8303,\n          0.8518112824778165,\n
0.8312722718433774\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df_metricas"}
```


7.2. Interpretación de resultados:

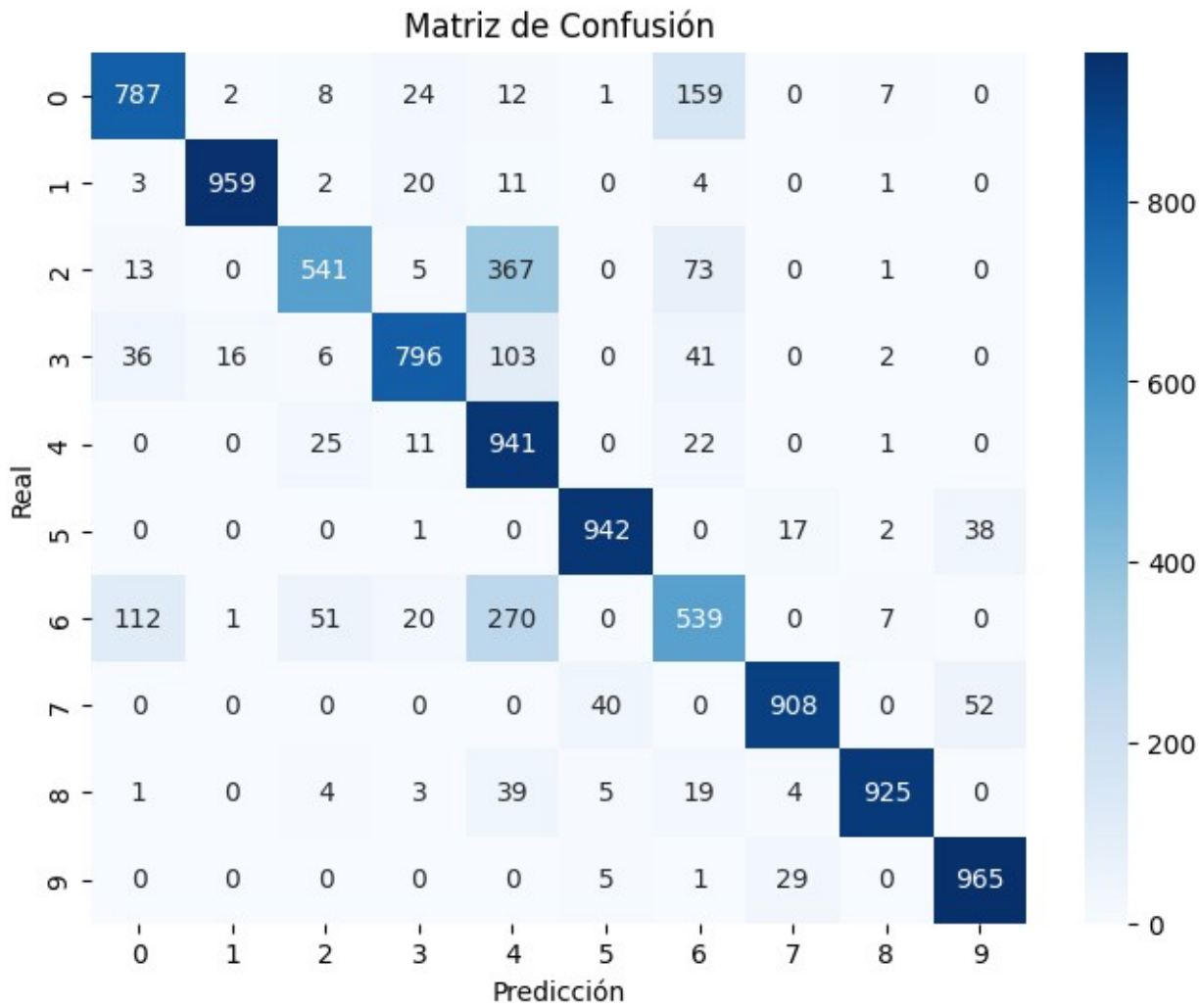
- El modelo presenta un rendimiento global cercano al 75%–80% en todas las métricas evaluadas. Se observa que la precisión, el recall y el F1-score mantienen valores similares entre sí, lo que indica un comportamiento equilibrado del modelo, sin sesgos marcados hacia clases específicas.

- La cercanía entre las métricas sugiere que el modelo no solo logra predecir correctamente una proporción considerable de instancias, sino que también mantiene un buen balance entre falsos positivos y falsos negativos.

- Las pequeñas variaciones en los resultados entre ejecuciones se deben a la naturaleza estocástica del entrenamiento del modelo, incluyendo la inicialización de pesos, el uso de Dropout y la división de los datos, lo cual es esperado en este tipo de modelos.

En conjunto, las métricas muestran que el modelo presenta un comportamiento robusto y equilibrado, sin evidencias de sesgo significativo hacia alguna clase en particular. La similitud entre precision, recall y F1-score confirma la estabilidad del modelo.

```python
from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicción")
plt.ylabel("Real")
plt.title("Matriz de Confusión")
plt.show()
```



7.3. Resultados Matriz de Confusión:

La matriz de confusión muestra que el modelo logra clasificar correctamente la mayoría de las clases, observándose una mayor concentración de valores en la diagonal principal. Sin embargo,

existen confusiones entre clases visualmente similares, como por ejemplo entre ciertas categorías de vestimenta (por ejemplo, camisetas y camisas), lo que indica que el modelo presenta dificultades para distinguir características sutiles entre ellas.

A pesar de estas confusiones, el modelo presenta un comportamiento general adecuado, con un equilibrio entre precisión y recall, lo que se ve reflejado en un F1-score cercano al accuracy obtenido.

#Parte 8: Ejemplos funcionamiento del modelo.

```
from sklearn.metrics import (
    confusion_matrix, classification_report,
    accuracy_score, precision_score, recall_score, f1_score,
    balanced_accuracy_score
)

acc = accuracy_score(y_test, y_pred)
prec_macro = precision_score(y_test, y_pred, average='macro')
rec_macro = recall_score(y_test, y_pred, average='macro')
f1_macro = f1_score(y_test, y_pred, average='macro')

prec_weighted = precision_score(y_test, y_pred, average='weighted')
rec_weighted = recall_score(y_test, y_pred, average='weighted')
f1_weighted = f1_score(y_test, y_pred, average='weighted')

balanced_acc = balanced_accuracy_score(y_test, y_pred)

print("Accuracy:", acc)
print("Precision (macro):", prec_macro)
print("Recall (macro):", rec_macro)
print("F1-score (macro):", f1_macro)
print("Precision (weighted):", prec_weighted)
print("Recall (weighted):", rec_weighted)
print("F1-score (weighted):", f1_weighted)
print("Balanced Accuracy:", balanced_acc)
```

```
Accuracy: 0.8303
Precision (macro): 0.8518112824778165
Recall (macro): 0.8303
F1-score (macro): 0.8312722718433774
Precision (weighted): 0.8518112824778163
Recall (weighted): 0.8303
F1-score (weighted): 0.8312722718433774
Balanced Accuracy: 0.8303
```

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.83      0.79      0.81      1000
           1       0.98      0.96      0.97      1000
           2       0.85      0.54      0.66      1000
           3       0.90      0.80      0.85      1000
           4       0.54      0.94      0.69      1000
           5       0.95      0.94      0.95      1000
           6       0.63      0.54      0.58      1000
           7       0.95      0.91      0.93      1000
           8       0.98      0.93      0.95      1000
           9       0.91      0.96      0.94      1000

    accuracy                           0.83     10000
   macro avg       0.85      0.83      0.83     10000
weighted avg       0.85      0.83      0.83     10000
```

- El análisis por clase muestra que el modelo presenta un rendimiento heterogéneo dependiendo de la categoría. Algunas clases, como la 1, 8 y 9, alcanzan métricas elevadas (superiores al 0.90 en precision y recall), lo que indica que el modelo logra identificarlas de manera consistente y con bajo nivel de error.
- En contraste, existen clases con un desempeño más bajo, como la clase 6 y la clase 2, donde tanto el recall como el F1-score disminuyen considerablemente. Esto sugiere que el modelo presenta dificultades para reconocer correctamente estas categorías, probablemente debido a similitudes visuales con otras clases.
- Un caso interesante es la clase 4, que presenta un recall alto pero una precisión más baja. Esto indica que el modelo logra identificar la mayoría de los elementos de esta clase, pero también comete errores al confundir otras clases como si pertenecieran a esta categoría.

Clases como 1, 8, 9 → fáciles → el modelo las reconoce bien.

Clases como 2 y 6 → difíciles → el modelo se confunde.

Clase 4 → detecta muchas, pero también se equivoca bastante.

```
# Elegimos 25 imágenes al azar
h, w = 5, 5
fig, axs = plt.subplots(h, w, figsize=(10,10))

for i in range(h):
    for j in range(w):
        idx = np.random.randint(len(X_test))

        img = X_test[idx].reshape(28,28)
        true_label = Y_test[idx]
```

```
        pred = model_final.predict(X_test[idx].reshape(1,784))
        pred_label = np.argmax(pred)

        axs[i,j].imshow(img, cmap='gray')
        axs[i,j].set_title(f"Real: {clases[true_label]}\nPred:
{clases[pred_label]}")
        axs[i,j].axis('off')
        color = "green" if true_label == pred_label else "red"

axs[i,j].set_title(
    f"R: {clases[true_label]}\nP: {clases[pred_label]}",
    color=color
)

plt.tight_layout()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step
```

Se visualizaron ejemplos de predicciones del modelo sobre el conjunto de prueba. Se observó que el modelo logra clasificar correctamente la mayoría de las prendas, especialmente aquellas con características visuales distintivas como bolsos y botines. Sin embargo, se identifican errores en clases visualmente similares, como camisas y camisetas, lo que evidencia una limitación del modelo para diferenciar detalles finos en las imágenes.

Conclusión final:

- La diferencia entre los resultados obtenidos en entrenamiento/validación y el conjunto de prueba se debe a que este último contiene datos completamente

nuevos, lo que representa un escenario más exigente para el modelo. Esto evidencia la capacidad de generalización del modelo en un contexto real.

- Cabe destacar que, al tratarse de una red neuronal tipo MLP, el modelo presenta limitaciones para capturar patrones espaciales complejos en imágenes, lo que explica por qué su rendimiento es menor en comparación con arquitecturas más avanzadas como redes convolucionales (CNN).